

Home Work 2

Distributed Systems

Assignment Structure

The homework is divided into three sections. Each team will be assigned **one question from each section**:

1. **Q1–Q3: Distributed Algorithms**
2. **Q4–Q6: Graph Algorithms**
3. **Q7–Q8: Real-World Applications**

General Instructions

- Use MPI to distribute the workload across processes.
- Programs must handle the stated constraints and required process counts.
- Submit the required source code, README, correctness verification, and performance results for each assigned question.
- Do not print debugging information as part of the required program output.

1. Distributed Algorithms

Q1. Distributed Matrix Multiplication using the Row-Row Method

You are given two matrices, A (of dimension $m \times n$) and B (of dimension $n \times p$). Implement an MPI program that computes $C = A \times B$ using the **Row-Row method**. In this method, matrix A is partitioned row-wise across processes, and matrix B is fully replicated (broadcast) to every process. Each row of C is computed as a weighted sum of the rows of B , where the weights are the entries of the corresponding row of A :

$$c_i = a_i[1] \cdot B[1, :] + a_i[2] \cdot B[2, :] + \cdots + a_i[n] \cdot B[n, :]$$

Input Constraints

- m, n, p range from small (e.g. 3×3 , for manual verification) to large (e.g. 1000×1000 or more, for timing/scaling)
- Matrices need not be square; test skewed shapes (tall $m \gg n$, or wide $n \gg m$)
- Test both cases: m divisible by P , and m not divisible by P
- Edge cases: $m = 1$ (single row), $n = 1$ (single column in A / single row in B)
- Entries are Integers.

Matrix Layout

- Master reads/generates A ($m \times n$) and B ($n \times p$).
- A is partitioned horizontally: each of P workers gets $\sim m/P$ rows (handle m not divisible by P).
- B is broadcast in full to all workers.

Computation

- Workers compute their assigned row-slices of C independently, without inter-worker communication.

Collection

- Master gathers all row-slices from workers and reconstructs C .

Example 1 — even split, $P = 3$

$$A = \begin{pmatrix} 1 & 2 \\ 0 & 3 \\ -1 & 4 \end{pmatrix}_{3 \times 2} \quad B = \begin{pmatrix} 2 & 3 & 4 \\ 1 & 0 & -1 \end{pmatrix}_{2 \times 3}$$

Distribution: P0 $\rightarrow$ row 1, P1 $\rightarrow$ row 2, P2 $\rightarrow$ row 3

$$\begin{aligned} c_1 &= 1 \cdot (2, 3, 4) + 2 \cdot (1, 0, -1) = (4, 3, 2) \\ c_2 &= 0 \cdot (2, 3, 4) + 3 \cdot (1, 0, -1) = (3, 0, -3) \\ c_3 &= -1 \cdot (2, 3, 4) + 4 \cdot (1, 0, -1) = (2, -3, -8) \end{aligned}$$

$$C = \begin{pmatrix} 4 & 3 & 2 \\ 3 & 0 & -3 \\ 2 & -3 & -8 \end{pmatrix}_{3 \times 3}$$

Example 2 — uneven split, $P = 3$

$$A = \begin{pmatrix} 1 & 0 \\ 2 & -1 \\ 0 & 3 \\ 1 & 1 \end{pmatrix}_{4 \times 2} \quad B = \begin{pmatrix} 1 & 2 \\ 0 & 1 \end{pmatrix}_{2 \times 2}$$

Distribution: P0 $\rightarrow$ 2 rows, P1 $\rightarrow$ 1 row, P2 $\rightarrow$ 1 row

$$c_1 = (1, 2) \quad c_2 = (2, 3) \quad c_3 = (0, 3) \quad c_4 = (1, 3)$$

$$C = \begin{pmatrix} 1 & 2 \\ 2 & 3 \\ 0 & 3 \\ 1 & 3 \end{pmatrix}_{4 \times 2}$$

Q2. Distributed Matrix Multiplication using the Column-Row Method

Question

Consider matrices A ($m \times n$) and B ($n \times p$), implement an MPI program that computes $C = A \times B$ using the **Column-Row method**.

Each column of A is paired with the corresponding row of B to form a rank-1 outer product (a full $m \times p$ partial matrix). The final result is the **sum** of all n such partial matrices:

$$C = \sum_{k=1}^n A[:, k] \cdot B[k, :]$$

Input Constraints

- m, n, p range from small (e.g. 3×3 , for manual verification) to large (e.g. 1000×1000 or more, for timing/scaling)
- Matrices need not be square; test skewed shapes (tall $m \gg n$, or wide $n \gg m$)
- Test both cases: n divisible by P , and n not divisible by P
- Edge cases: $n = 1$ (single column-row pair), $m = 1$ or $p = 1$
- Entries are integers.

Matrix Layout

- Master reads/generates A and B .
- A is partitioned column-wise, B row-wise correspondingly: each of P workers gets $\sim n/P$ column-row pairs (handle n not divisible by P).
- No full matrix is broadcast — each worker only gets its assigned columns of A and rows of B .

Computation

- Each worker computes the outer product for each assigned pair and sums them locally into one partial $m \times p$ matrix.

Collection

- Master **reduces (sums)** the partial matrices from all workers via `MPI_Reduce` or `MPI_SUM` — this is different from Row-Row, where results are gathered, not summed.

Example 1 — even split, $P = 2$

$$A = \begin{pmatrix} 1 & 2 \\ 0 & 3 \\ -1 & 4 \end{pmatrix}_{3 \times 2} \quad B = \begin{pmatrix} 2 & 3 & 4 \\ 1 & 0 & -1 \end{pmatrix}_{2 \times 3}$$

P0: column 1 of A + row 1 of B

$$\begin{pmatrix} 1 \\ 0 \\ -1 \end{pmatrix} \begin{pmatrix} 2 & 3 & 4 \end{pmatrix} = \begin{pmatrix} 2 & 3 & 4 \\ 0 & 0 & 0 \\ -2 & -3 & -4 \end{pmatrix}$$

P1: column 2 of A + row 2 of B

$$\begin{pmatrix} 2 \\ 3 \\ 4 \end{pmatrix} \begin{pmatrix} 1 & 0 & -1 \end{pmatrix} = \begin{pmatrix} 2 & 0 & -2 \\ 3 & 0 & -3 \\ 4 & 0 & -4 \end{pmatrix}$$

Sum (reduce):

$$C = \begin{pmatrix} 4 & 3 & 2 \\ 3 & 0 & -3 \\ 2 & -3 & -8 \end{pmatrix}_{3 \times 3}$$

Example 2 — uneven split, $P = 2$

$$A = \begin{pmatrix} 1 & 0 & 2 \\ 3 & 1 & 0 \end{pmatrix}_{2 \times 3} \quad B = \begin{pmatrix} 1 & 2 \\ 0 & 1 \\ 2 & 0 \end{pmatrix}_{3 \times 2}$$

P0: columns 1 & 2 of A + rows 1 & 2 of B (sums 2 outer products locally)

$$\begin{pmatrix} 1 \\ 3 \end{pmatrix} \begin{pmatrix} 1 & 2 \end{pmatrix} + \begin{pmatrix} 0 \\ 1 \end{pmatrix} \begin{pmatrix} 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 2 \\ 3 & 7 \end{pmatrix}$$

P1: column 3 of A + row 3 of B (1 outer product)

$$\begin{pmatrix} 2 \\ 0 \end{pmatrix} \begin{pmatrix} 2 & 0 \end{pmatrix} = \begin{pmatrix} 4 & 0 \\ 0 & 0 \end{pmatrix}$$

Reduce (sum):

$$C = \begin{pmatrix} 5 & 2 \\ 3 & 7 \end{pmatrix}_{2 \times 2}$$

Q3. Bitonic Sort

Question

You are given a sequence of N elements, split equally across P processes. Implement an MPI program that sorts the sequence using **Bitonic Sort**.

Each process first sorts its own local chunk. Then, over several stages, pairs of processes exchange their chunks, compare-exchange elements, and each process keeps its correct half (smaller or larger, depending on the stage). After all stages, the chunks are gathered and concatenated to form the final sorted sequence.

Note : Please go through below examples and link to correctly understand the sorting method before implementation - [link1](#)

Input Constraints

- N and P must both be powers of 2, with N divisible by P
- N ranges from small (e.g. $N = 8$, for manual verification) to large (e.g. $N = 10^6$ or more, for timing/scaling)
- Edge cases: $N = P$ (1 element per process), $P = 1$ (reduces to a purely local sort)
- Elements are integers.

Example 1 — $N = 4, P = 2$

Input: (4, 1, 3, 2)

Distribute: $P_0 = (4, 1), P_1 = (3, 2)$

Local sort: P_0 (asc) $\rightarrow (1, 4), P_1$ (desc) $\rightarrow (3, 2)$ — combined (1, 4, 3, 2) is bitonic

Stage 1 ($P_0 \leftrightarrow P_1$ exchange, compare position-wise):

pos 0 : 1 vs 3 $\rightarrow$ low = 1, high = 3
pos 1 : 4 vs 2 $\rightarrow$ low = 2, high = 4

P_0 (low) = (1, 2), P_1 (high) = (3, 4)

Re-sort locally: $P_0 = (1, 2), P_1 = (3, 4)$

Output (gather): (1, 2, 3, 4)

Example 2 — $N = 8, P = 2$

Input: (5, 3, 8, 1, 9, 2, 7, 4)

Distribute: $P_0 = (5, 3, 8, 1), P_1 = (9, 2, 7, 4)$

Local sort: P_0 (asc) $\rightarrow (1, 3, 5, 8), P_1$ (desc) $\rightarrow (9, 7, 4, 2)$ — combined sequence is bitonic

Stage 1 ($P_0 \leftrightarrow P_1$ exchange, compare position-wise):

pos 0 : 1 vs 9 $\rightarrow$ low = 1, high = 9
pos 1 : 3 vs 7 $\rightarrow$ low = 3, high = 7
pos 2 : 5 vs 4 $\rightarrow$ low = 4, high = 5
pos 3 : 8 vs 2 $\rightarrow$ low = 2, high = 8

P_0 (low) = (1, 3, 4, 2), P_1 (high) = (9, 7, 5, 8)

Re-sort locally: $P_0 = (1, 2, 3, 4)$, $P_1 = (5, 7, 8, 9)$

Output (gather): (1, 2, 3, 4, 5, 7, 8, 9)

Deliverables for this section

- MPI source code for the assigned algorithm.
- Program must run correctly for $P = 1, 2, 4, 8$ processes.
- README with compilation and execution instructions.
- Correctness verification against a sequential computation.
- Execution times for $P = 1, 2, 4, 8$ over varying input sizes.
- Speedup/efficiency plots and a short analysis of communication versus computation.

2. Graph Algorithms

Q4. Triangle Counting in an Undirected Graph

- Implement a distributed algorithm to count the number of triangles in a given undirected graph.
- The input is an undirected graph.
- Each machine stores an equal-sized subset of distinct edges. The algorithm must use MPI to communicate and efficiently find the total global count of triangles.
- Ensure no double-counting of triangles occurs across different processes.

Constraints

- Number of vertices (V): $3 \leq V \leq 10^5$
- Number of edges (E): $3 \leq E \leq 10^6$
- Vertices are 0-indexed.

Input Format

- First line: Two integers V and $E \rightarrow$ number of vertices and edges.
- Next E lines: Two integers u and v representing an undirected edge between vertex u and vertex v .

Output Format

- A single integer representing the total global count of triangles in the graph.

Sample Input

```
4 5
0 1
1 2
2 0
2 3
3 0
```

Sample Output

```
2
```

(Explanation: The two triangles are formed by vertices $\{0, 1, 2\}$ and $\{0, 2, 3\}$)

Q5. All-Pairs Shortest Path using Floyd-Warshall

- Find the shortest distance between all pairs of nodes in a weighted graph using the Floyd-Warshall algorithm.
- Distribute the rows of the distance matrix across MPI processes.

Constraints

- Number of vertices (N): $2 \leq N \leq 1000$
- Edge weights (w): $-1000 \leq w \leq 1000$
- The graph is guaranteed to have no negative weight cycles.
- A value of 1000000 in the input represents ∞ (no direct edge).

Input Format

- First line: Two integers V and $E \rightarrow$ number of vertices and edges.
- Next E lines: Three integers u , v and w representing a directed edge from vertex u to vertex v with weight w .

Output Format

- Print N lines, each containing N space-separated integers, representing the shortest distance between every pair of nodes.

Sample Input

```
3 3
0 1 5
1 2 3
2 0 2
```

Sample Output

```
0 5 8
5 0 3
2 7 0
```

Q6. Connected Components of a Large Graph

- Identify the connected components of a large undirected graph.
- The graph is provided as an adjacency list. Distribute the vertices and their adjacency lists across the processes. Use parallel message-passing techniques to assign each vertex a component ID.

Constraints

- Number of vertices (V): $1 \leq V \leq 10^5$
- Number of edges (E): $0 \leq E \leq 10^6$
- Vertices are 0-indexed.
- The component ID of a vertex must be the minimum vertex ID present within that connected component.

Input Format

- First line: A single integer $V \rightarrow$ number of vertices.
- Next V lines: The adjacency list of the graph. The i -th line (where $0 \leq i < V$) has the format:

$k \quad v_1 \quad v_2 \dots v_k$

where:

- k = number of neighbors for vertex i
- v_j = neighbor vertex ID

Output Format

- Print V lines. Each line should contain two integers: `vertex_id` `component_id`.
- The output must be sorted by `vertex_id` in ascending order.

Sample Input

```
5
1 1
1 0
2 3 4
1 2
1 2
```

Sample Output

```
0 0
1 0
2 2
3 2
4 2
```

(Explanation: Vertices 0 and 1 are connected, forming a component with ID 0. Vertices 2, 3, and 4 are connected, forming a component with ID 2).

Deliverables for this section

- MPI source code for the assigned algorithm.
- Program must run correctly for $P = 1, 2, 4, 8$ processes.
- README with compilation and execution instructions.

- Correctness verification against a sequential computation.
- Execution times for $P = 1, 2, 4, 8$ over varying input sizes.
- Speedup/efficiency plots and a short analysis of communication versus computation.

3. Real-World Applications

Each team assigned this section will implement one of the following real-world data analytics problems using MPI. Submit both a sequential and MPI implementation.

Q7. Large-Scale Server Log Analytics

Input

First line:

```
N K S
```

Next N lines:

```
timestamp server_id endpoint_id user_id status_code response_time bytes_sent
```

A request is successful if `status_code < 400`.

Required Computations

- Total, successful, and failed requests.
- Average, minimum, and maximum response time.
- Total bytes sent and counts of 2xx, 3xx, 4xx, and 5xx responses.
- Top- K servers by request count, with average response time.
- Top- K endpoints by request count, with total bytes.
- Busiest 60-second interval.

Output

```
TOTAL_REQUESTS <value>
SUCCESSFUL_REQUESTS <value>
FAILED_REQUESTS <value>
AVERAGE_RESPONSE_TIME <value>
MIN_RESPONSE_TIME <value>
MAX_RESPONSE_TIME <value>
TOTAL_BYTES <value>
STATUS_2XX <value>
STATUS_3XX <value>
STATUS_4XX <value>
STATUS_5XX <value>
BUSIEST_INTERVAL <interval_id> <count>
TOP_SERVERS
<server_id> <request_count> <average_response_time>
...
TOP_ENDPOINTS
<endpoint_id> <request_count> <total_bytes>
...
```

Top- K results are sorted by decreasing count, then increasing ID. Use `timestamp / 60` for the interval ID.

Q8. Large-Scale Weather and Environmental Data Analytics

Input

First line:

```
N K S
```

Next N lines:

```
timestamp station_id temperature humidity pressure rainfall wind_speed
```

Required Computations

- Total measurements.
- Average, minimum, and maximum temperature, humidity, and pressure.
- Total and maximum rainfall; average and maximum wind speed.
- Top- K stations by measurement count, with average temperature and total rainfall.
- Hottest and coldest measurements.
- Busiest 60-second interval.
- Number of measurements with temperature ≥ 40.0 or ≤ 0.0 .

Top- K stations are sorted by decreasing count, then increasing station ID. For hottest/coldest measurements, ties use smaller timestamp, then station ID.

Output

```
TOTAL_MEASUREMENTS <value>
AVERAGE_TEMPERATURE <value>
MIN_TEMPERATURE <value>
MAX_TEMPERATURE <value>
AVERAGE_HUMIDITY <value>
MIN_HUMIDITY <value>
MAX_HUMIDITY <value>
AVERAGE_PRESSURE <value>
MIN_PRESSURE <value>
MAX_PRESSURE <value>
TOTAL_RAINFALL <value>
MAX_RAINFALL <value>
AVERAGE_WIND_SPEED <value>
MAX_WIND_SPEED <value>
EXTREME_TEMPERATURE_EVENTS <value>
HOTTEST_MEASUREMENT <temperature> <station_id> <timestamp>
COLDEST_MEASUREMENT <temperature> <station_id> <timestamp>
BUSIEST_INTERVAL <interval_id> <count>
TOP_STATIONS
<station_id> <measurement_count> <average_temperature> <total_rainfall>
...
```

Dataset Generation and Benchmarking

The supplied examples are for correctness testing. Generate larger reproducible datasets using a fixed seed and document the generation method, parameters, sizes, and assumptions.

Benchmark both implementations using multiple input sizes and MPI process counts. Report execution time, speedup/efficiency, and a short analysis of computation, communication, data distribution, and scalability.

Deliverables for this section

- Sequential implementation, MPI implementation, and dataset generator.
- README with compilation, execution, generation, and benchmark instructions.
- Correctness verification using the provided samples.
- Reproducible dataset-generation code/procedure.
- Benchmark results for multiple input sizes and process counts.
- Speedup/efficiency plots and a short performance analysis.